Color Grading Application Documentation

Split-package version — User guide first, then technical reference with file locations

This manual describes the split multi-file version of the application. It starts with a practical user guide and then moves into a technical reference. Every function, class, and method includes the file where it lives so the code is easier to navigate and maintain.

Part I — User Guide

1. What the application does

The application is a desktop image-grading tool for PNG, JPG, and JPEG files. It follows a non-destructive, parameter-driven editing model: the source image is preserved, the current edit is stored as state, and the preview is rebuilt from that state instead of repeatedly rewriting the original image.

Libraries used in the split version:

- PySide6 for the desktop interface, custom widgets, actions, timers, dialogs, graphics-view display, and the worker-thread render flow.
- NumPy for RGBA image arrays, histogram generation, lookup-table work, tonal masks, channel math, and lightweight preview hashing.
- Pillow (PIL) for image loading, RGBA conversion, and fallback resizing when OpenCV is unavailable.
- OpenCV, when installed, for faster resizing and faster LUT application in the preview pipeline.
- Python standard-library modules such as copy, json, os, sys, dataclasses, and typing for cloning, persistence, state management, and packaging.

Architecturally, the app is built around a staged processing pipeline rather than direct destructive edits.

- The original image is normalized to contiguous RGBA NumPy format when it is opened.
- Geometry changes are applied in a stable order: rotate, flip, crop, and then resize.
- Color work happens after geometry, using LUTs plus float-space channel adjustments.
- Interactive edits use a debounced fast-preview path so sliders and curves stay more responsive.
- A separate full-quality pass refreshes histogram data after the user pauses.
- A render generation counter prevents stale worker results from overwriting newer preview states.
- Undo and redo are state-snapshot based, which is much lighter than storing a full bitmap for every micro-edit.

2. Package layout

File	Role	Why it exists
config.py	Shared constants	Keeps filters, render timings, preview sizing, and other common constants in one place.
utils.py	General helpers	Holds conversion, resize, histogram, LUT, save-path, and aspect-fit helpers.
transforms.py	Coordinate mapping	Keeps crop geometry stable across rotation and flips.
models.py	Session/state definitions	Defines dataclasses for curves, tones, crop, resize, adjustment state, and history.
processing.py	Image pipeline	Applies geometry, resize, and color work to image arrays.
render.py	Worker-side rendering	Moves preview rendering out of the busiest part of the UI thread.
widgets/curve_editor.py	Curve widget	Interactive curve editing with histogram overlay.
widgets/histogram_widget.py	Histogram widget	Draws a standalone histogram panel.
widgets/image_view.py	Image canvas widget	Handles display, crop overlay, zoom, pan, and drag-and-drop.
main_window.py	Application controller	Connects widgets, state, persistence, history, and rendering.
main.py	Entry point	Creates QApplication and launches the main window.

3. Typical editing workflow

1. Open an image with Ctrl+O, the toolbar, or drag-and-drop into the viewer.
2. Start with broad corrections such as exposure, contrast, gamma, temperature, tint, and white-balance strength.
3. Refine with per-channel intensity, tonal-region RGB adjustments, and curves.
4. Use the histogram and the curve background overlay to monitor clipping and channel balance.
5. Apply crop, rotation, flips, and resize once the color work is close to final.
6. Compare the edited view with the original before exporting.
7. Export the image or save a preset/project if you want to continue later.

Part II — Technical Reference

4. File-by-file reference

config.py

This file centralizes shared constants so the rest of the package can import one stable set of names.

Constants block

- Location: config.py

- What it does: Defines shared configuration values used throughout the package.

- How it works: It centralizes user-visible strings, supported file filters, render timing constants, histogram sizing, preview oversampling, and minimum crop size so other files can import the same values consistently.
- Output / effect: Read-only configuration used by the rest of the package.

utils.py
This file contains general-purpose helper functions used by both the UI layer and the processing layer.

pil_to_numpy_rgba(img)
- Location: utils.py
- What it does: Converts a Pillow image into the application's internal RGBA NumPy format.
- How it works: It forces RGBA mode, converts the pixels to a uint8 NumPy array, and makes the array contiguous in memory so later processing and Qt conversion are stable.
- Output / effect: Returns a contiguous uint8 RGBA array.

numpy_to_qimage(arr)
- Location: utils.py
- What it does: Converts a NumPy image array into a QImage for display.
- How it works: It inspects the channel count, chooses the matching Qt image format, builds a QImage over the underlying memory, and then copies it so the Qt object remains valid if the Python array changes later.
- Output / effect: Returns a detached QImage.

numpy_to_pixmap(arr)
- Location: utils.py
- What it does: Builds a QPixmap from a NumPy array.
- How it works: It is a small wrapper around numpy_to_qimage followed by QPixmap.fromImage.
- Output / effect: Returns a QPixmap.

resize_rgba(arr, size, fast=True)
- Location: utils.py
- What it does: Resizes an RGBA image array to a target width and height.
- How it works: It skips work when the target size already matches the input. If OpenCV is available it uses cv2.resize with fast or high-quality interpolation. Otherwise it falls back to Pillow resizing.
- Output / effect: Returns a resized RGBA array.
- Notes: This is one of the package's most performance-sensitive helpers.

downscale_rgba(arr, max_side)
- Location: utils.py

- What it does: Shrinks an image so its longest side does not exceed a limit.
- How it works: It measures the current longest dimension, computes a scale factor only when needed, and then calls resize_rgba with the reduced size.
- Output / effect: Returns either the original array or a downscaled copy.

hash_array(arr)

- Location: utils.py
- What it does: Creates a lightweight preview identity token.
- How it works: Instead of hashing every pixel, it samples the array on a coarse grid and sums sampled RGB values. That makes it cheap enough for UI refresh checks.
- Output / effect: Returns an integer summary value.
- Notes: This is a UI helper, not a cryptographic hash.

fit_size_preserving_aspect(src_w, src_h, max_w, max_h)

- Location: utils.py
- What it does: Computes a fitted width and height while preserving aspect ratio.
- How it works: It clamps minimum dimensions, computes the width and height scale factors, takes the limiting one, and returns rounded integer dimensions.
- Output / effect: Returns a (width, height) tuple.

unique_path(path)

- Location: utils.py
- What it does: Finds the next available non-conflicting filename in a directory.
- How it works: It inspects the target directory, recognizes siblings with the same stem and extension plus numeric suffixes like (1) and (2), and returns the next free path.
- Output / effect: Returns a path string.
- Notes: The numbering comes from actual directory contents, not from an in-memory counter.

choose_save_path(parent, title, suggested, file_filter)

- Location: utils.py
- What it does: Opens a save dialog that already suggests a unique filename.
- How it works: It turns the suggested path into an absolute path, computes the next free name with unique_path, pre-fills the dialog, and disables overwrite confirmation so the application can keep its own naming behavior.
- Output / effect: Returns a chosen path string or an empty string.

build_curve_lut(points)

- Location: utils.py
- What it does: Converts curve control points into a 256-entry lookup table.
- How it works: It clamps points to the 0–1 domain, sorts them by x-value, guarantees the endpoints exist, interpolates across 256 samples, and maps the result into uint8 output values.
- Output / effect: Returns a uint8 LUT array.
- Notes: This is the bridge between the curve editor widget and the processing pipeline.

compose_scalar_lut(brightness, contrast, gamma, exposure)

- Location: utils.py
- What it does: Creates the lookup table used for global scalar tonal adjustments.
- How it works: It starts from normalized values, applies exposure scaling, brightness shift, contrast around the midpoint, and gamma correction, then maps back into 0–255 integer output values.
- Output / effect: Returns a uint8 LUT array.

histogram_from_rgba(arr)

- Location: utils.py
- What it does: Builds red, green, and blue histograms from an image array.
- How it works: It first downsizes the image for efficiency, strips alpha, and runs NumPy histogram calculations separately for each RGB channel.
- Output / effect: Returns a dictionary with r, g, and b histogram arrays.

transforms.py

This file isolates transform and coordinate-conversion math used to keep crop behavior stable.

transformed_size(width, height, rotation)

- Location: transforms.py
- What it does: Computes the dimensions of an image after rotation.
- How it works: For 90-degree and 270-degree rotations width and height swap. For 0-degree and 180-degree rotations they stay in the same order.
- Output / effect: Returns transformed width and height.

transform_point_original_to_view(x, y, width, height, rotation, flip_h, flip_v)

- Location: transforms.py
- What it does: Maps one point from original-image space into transformed view space.
- How it works: It applies rotation first, tracks the transformed bounds, and then applies optional horizontal and vertical flips inside that transformed coordinate system.
- Output / effect: Returns transformed x and y coordinates.

inverse_transform_point_view_to_original(x, y, width, height, rotation, flip_h, flip_v)

- Location: transforms.py
- What it does: Maps one point from transformed view space back into original-image space.
- How it works: It reverses flips first and then applies the inverse mapping for the selected rotation.
- Output / effect: Returns original-image x and y coordinates.

rect_view_to_original(rect, width, height, rotation, flip_h, flip_v)

- Location: transforms.py
- What it does: Converts a rectangle from transformed-view coordinates into original-image coordinates.
- How it works: It maps the rectangle corners with the inverse point transform, computes the bounding box of the mapped points, and clamps the result to valid original-image bounds.
- Output / effect: Returns a QRect in original-image coordinates.

rect_original_to_view(rect, width, height, rotation, flip_h, flip_v)

- Location: transforms.py
- What it does: Converts a rectangle from original-image coordinates into transformed-view coordinates.
- How it works: It transforms each corner into rotated/flipped space, builds a bounding box, and clamps the result to valid transformed bounds.
- Output / effect: Returns a QRect in transformed-view coordinates.

models.py

This file defines the data structures that represent the editable session and its history.

Class CurveSet

- Location: models.py
- What it does: Stores the four curve definitions used by the color pipeline.
- How it works: It keeps separate point lists for the master, red, green, and blue curves. The default for each is the identity line.
- Output / effect: Acts as part of the editable session state.

CurveSet.to_json()

- Location: models.py
- What it does: Serializes all four curves.
- How it works: It converts point tuples into plain lists so the data can be written to JSON-based preset and project files.
- Output / effect: Returns a dictionary.

CurveSet.from_json(data)

- Location: models.py
- What it does: Rebuilds a CurveSet from stored data.
- How it works: It restores each channel's point list and falls back to identity curves when data is missing.
- Output / effect: Returns a CurveSet instance.

Class ToneRGB

- Location: models.py
- What it does: Stores one three-channel tonal adjustment group.
- How it works: It is used for shadows, midtones, and highlights. Each instance contains separate red, green, and blue float offsets.
- Output / effect: Acts as a compact tonal-control container.

ToneRGB.to_json()

- Location: models.py
- What it does: Serializes one tonal RGB group.
- How it works: It converts the dataclass fields into a plain dictionary.
- Output / effect: Returns a dictionary.

ToneRGB.from_json(data)

- Location: models.py
- What it does: Rebuilds a tonal RGB group from stored data.
- How it works: It reads r, g, and b values from a dictionary and converts them to floats.
- Output / effect: Returns a ToneRGB instance.

Class CropRect

- Location: models.py
- What it does: Stores crop geometry and whether cropping is active.
- How it works: It contains x, y, width, height, and an enabled flag. The rectangle is interpreted in transformed pre-crop space.
- Output / effect: Acts as the persistent crop state.

CropRect.to_json()

- Location: models.py
- What it does: Serializes crop state.
- How it works: It converts the dataclass into a plain dictionary for preset and project storage.
- Output / effect: Returns a dictionary.

CropRect.from_json(data)

- Location: models.py
- What it does: Rebuilds crop state from stored data.
- How it works: It reads integer coordinates and the enabled flag from a dictionary.
- Output / effect: Returns a CropRect instance.

Class ResizeState

- Location: models.py
- What it does: Stores resize dimensions and whether resize is active.
- How it works: It contains width, height, and an enabled flag so the processing pipeline can treat resize as an optional stage.
- Output / effect: Acts as the persistent resize state.

ResizeState.to_json()

- Location: models.py
- What it does: Serializes resize state.
- How it works: It converts the dataclass into a plain dictionary.
- Output / effect: Returns a dictionary.

ResizeState.from_json(data)

- Location: models.py
- What it does: Rebuilds resize state from stored data.
- How it works: It restores width, height, and enabled values from a dictionary.
- Output / effect: Returns a ResizeState instance.

Class AdjustmentState

- Location: models.py
- What it does: Represents the full editable session state.
- How it works: It gathers scalar grading values, tonal controls, geometry transforms, crop and resize state, and the four curve definitions into one object.
- Output / effect: Acts as the central state model.

AdjustmentState.clone()

- Location: models.py
- What it does: Creates a deep copy of the session state.
- How it works: It uses deepcopy so nested dataclasses such as curves and tonal controls are copied safely without shared references.

- Output / effect: Returns a cloned AdjustmentState.

AdjustmentState.to_json()

- Location: models.py
- What it does: Serializes the full session state.
- How it works: It exports scalar, tonal, geometry, crop, resize, and curve values into a single dictionary suitable for JSON storage.
- Output / effect: Returns a dictionary.

AdjustmentState.from_json(data)

- Location: models.py
- What it does: Rebuilds the full session state from stored data.
- How it works: It restores scalar values, nested dataclasses, rotation, flips, crop state, resize state, and curves while applying defaults where fields are missing.
- Output / effect: Returns an AdjustmentState instance.

Class HistoryManager

- Location: models.py
- What it does: Manages undo and redo stacks.
- How it works: It stores cloned AdjustmentState snapshots rather than full image bitmaps, which keeps history lighter and simpler.
- Output / effect: Acts as the parameter-history controller.

HistoryManager.__init__()

- Location: models.py
- What it does: Initializes the undo and redo stacks.
- How it works: It starts both stacks as empty lists.
- Output / effect: Creates the history manager.

HistoryManager.clear()

- Location: models.py
- What it does: Removes all undo and redo history.
- How it works: It empties both stacks, usually when a new image is loaded.
- Output / effect: Resets history.

HistoryManager.push(state)

- Location: models.py
- What it does: Adds a new state to undo history when it differs from the latest one.

- How it works: It compares the JSON form of the newest state with the previous one to avoid duplicate consecutive snapshots, then clears redo history because a new edit branch has been created.
- Output / effect: Updates the undo/redo stacks.

HistoryManager.can_undo()

- Location: models.py
- What it does: Reports whether undo is available.
- How it works: It checks whether there is more than one snapshot on the undo stack.
- Output / effect: Returns True or False.

HistoryManager.can_redo()

- Location: models.py
- What it does: Reports whether redo is available.
- How it works: It checks whether the redo stack contains any snapshots.
- Output / effect: Returns True or False.

HistoryManager.undo(current)

- Location: models.py
- What it does: Moves one step backward in history.
- How it works: It moves the latest undo state to the redo stack and returns the new current undo state.
- Output / effect: Returns the state to restore.

HistoryManager.redo(current)

- Location: models.py
- What it does: Moves one step forward in history.
- How it works: It pops a state from redo, pushes it back onto undo, and returns it as the new active state.
- Output / effect: Returns the state to restore.

processing.py

This file contains the actual image-processing stages used by preview and export.

Class ImageProcessor

- Location: processing.py
- What it does: Groups the core image-processing operations used by preview and export.
- How it works: It separates geometry, resize, and color work into explicit static methods so the render pipeline stays readable.
- Output / effect: Acts as the central processing layer.

ImageProcessor.apply_crop_rotate_flip(img, state)

- Location: processing.py
- What it does: Applies geometry transforms before the color stage.
- How it works: It rotates first, then flips, then applies the crop rectangle. This order keeps crop coordinates aligned with the same transformed space the user sees in the viewer.
- Output / effect: Returns the transformed RGBA image array.

ImageProcessor.apply_resize(img, state, fast)

- Location: processing.py
- What it does: Applies the optional resize stage.
- How it works: It checks whether resize is enabled and whether width and height are valid, then calls the shared resize helper.
- Output / effect: Returns either the resized image or the original image.

ImageProcessor.apply_color(img, state, skip_tonal=False)

- Location: processing.py
- What it does: Applies the color-grading pipeline.
- How it works: It builds scalar and curve LUTs, applies them to the RGB image, converts to float space, applies white balance, tint, per-channel intensity, and optional shadows/midtones/highlights adjustments, then converts back to uint8 RGBA.
- Output / effect: Returns the graded RGBA image array.
- Notes: This is the heart of the image look-creation stage.

render.py

This file contains the worker-side rendering classes.

Class RenderRequest

- Location: render.py
- What it does: Stores everything needed for one render pass.
- How it works: It captures the generation number, source image, adjustment state, target display size, quality mode, and whether tonal work may be skipped.
- Output / effect: Acts as a transport object between the UI and the worker.

RenderRequest.__init__(...)

- Location: render.py
- What it does: Initializes a render request object.
- How it works: It stores all render inputs so the worker can process them later.

- Output / effect: Creates a request snapshot.

Class RenderWorker

- Location: render.py
- What it does: Processes render requests and emits preview results.
- How it works: It is a QObject so it can live on a QThread and send Qt signals safely back to the UI.
- Output / effect: Acts as the asynchronous preview worker.

RenderWorker.__init__()

- Location: render.py
- What it does: Initializes worker state.
- How it works: It sets the pending request slot and the busy flag used for request coalescing.
- Output / effect: Creates the worker object.

RenderWorker.submit(request)

- Location: render.py
- What it does: Queues a render request.
- How it works: It stores only the newest pending request and starts processing immediately if the worker is idle.
- Output / effect: Begins or updates render scheduling.

RenderWorker._process_next()

- Location: render.py
- What it does: Processes the next pending request.
- How it works: It applies geometry, resize, preview fitting, and color grading. It then emits the rendered image, and for full-quality passes it also emits histogram data. If a newer request arrived while rendering, it continues with that one immediately.
- Output / effect: Emits preview and histogram results.
- Notes: This coalescing behavior is one of the key responsiveness optimizations.

widgets/histogram_widget.py

This file contains the standalone histogram display widget.

Class HistogramWidget

- Location: widgets/histogram_widget.py
- What it does: Draws a standalone RGB histogram panel.
- How it works: It stores the latest histogram arrays and repaints whenever the histogram data changes.
- Output / effect: Acts as the histogram display widget.

HistogramWidget.__init__(parent=None)

- Location: widgets/histogram_widget.py
- What it does: Initializes the histogram widget.
- How it works: It sets a minimum height and starts with zeroed histogram arrays for red, green, and blue.
- Output / effect: Creates the widget.

HistogramWidget.set_histogram(hist)

- Location: widgets/histogram_widget.py
- What it does: Loads histogram data into the widget.
- How it works: It stores the histogram dictionary and calls update so paintEvent will redraw the graph.
- Output / effect: Refreshes the widget.

HistogramWidget.paintEvent(event)

- Location: widgets/histogram_widget.py
- What it does: Draws the histogram panel contents.
- How it works: It paints the background, border, scales channel values to the current maximum, and draws filled area graphs for each RGB channel.
- Output / effect: Renders the histogram panel.

widgets/curve_editor.py

This file contains the interactive curve editor widget.

Class CurveEditor

- Location: widgets/curve_editor.py
- What it does: Provides interactive curve editing with histogram overlay.
- How it works: It stores curve points, the active channel, drag state, and histogram data, then renders a square plotting area with draggable handles.
- Output / effect: Acts as the curve editor widget.

CurveEditor.__init__(parent=None)

- Location: widgets/curve_editor.py
- What it does: Initializes the curve editor widget.
- How it works: It prepares default identity points, sets widget sizing, enables mouse tracking, and creates storage for the active channel and histogram arrays.
- Output / effect: Creates the widget.

CurveEditor.set_channel(channel)

- Location: widgets/curve_editor.py

- What it does: Changes which curve is displayed and edited.
- How it works: It stores the selected channel and repaints the widget.
- Output / effect: Refreshes the active curve.

CurveEditor.set_points(points)

- Location: widgets/curve_editor.py
- What it does: Loads a new set of control points.
- How it works: It sorts the list by x-position and schedules a redraw.
- Output / effect: Updates the curve shape.

CurveEditor.set_histogram(hist)

- Location: widgets/curve_editor.py
- What it does: Supplies histogram data for the curve background.
- How it works: It stores the histogram arrays and repaints so the background reflects the latest tonal distribution.
- Output / effect: Refreshes the histogram overlay.

CurveEditor._content_rect()

- Location: widgets/curve_editor.py
- What it does: Computes the square graph area inside the widget.
- How it works: It takes the smaller of the available width and height after margins so the curve plot remains square.
- Output / effect: Returns a QRectF.

CurveEditor._to_widget(p)

- Location: widgets/curve_editor.py
- What it does: Converts normalized curve coordinates into widget coordinates.
- How it works: It maps 0–1 x/y values into the square content rectangle and flips the y-axis so larger values appear higher on screen.
- Output / effect: Returns a QPointF.

CurveEditor._to_normalized(pos)

- Location: widgets/curve_editor.py
- What it does: Converts a mouse position into normalized graph coordinates.
- How it works: It maps widget-space cursor coordinates back into 0–1 graph space and clamps both axes to valid limits.
- Output / effect: Returns normalized x and y values.

CurveEditor._find_handle(pos)

- Location: widgets/curve_editor.py
- What it does: Detects whether the mouse is near an existing point handle.
- How it works: It checks the distance between the cursor and each rendered point handle and returns the matching index when one is close enough.
- Output / effect: Returns a point index or None.

CurveEditor.mousePressEvent(event)

- Location: widgets/curve_editor.py
- What it does: Starts a drag, inserts a point, or removes a point depending on the mouse action.
- How it works: Left-click selects an existing point or adds a new one. Right-click removes a non-endpoint point. The widget emits pointsChanged immediately for live preview updates.
- Output / effect: Begins curve editing.

CurveEditor.mouseMoveEvent(event)

- Location: widgets/curve_editor.py
- What it does: Moves the currently dragged point.
- How it works: It converts the cursor to normalized graph space, keeps endpoint x-values fixed, preserves ordering for interior points, updates the active point, and emits pointsChanged.
- Output / effect: Updates the active curve live.

CurveEditor.mouseReleaseEvent(event)

- Location: widgets/curve_editor.py
- What it does: Ends point dragging.
- How it works: It clears the drag index and emits dragFinished so the main window can finalize history and render state.
- Output / effect: Completes the edit gesture.

CurveEditor.paintEvent(event)

- Location: widgets/curve_editor.py
- What it does: Draws the curve editor contents.
- How it works: It paints the background, histogram overlay, grid, border, active curve line, and draggable point handles using the color associated with the selected channel.
- Output / effect: Renders the widget.

widgets/image_view.py

This file contains the image-canvas widget responsible for display and crop interaction.

Class ImageView

- Location: widgets/image_view.py
- What it does: Displays the preview image and manages interactive crop editing.
- How it works: It subclasses QGraphicsView so the image can be zoomed, panned, and drawn over with crop overlays and handles.
- Output / effect: Acts as the image canvas widget.

ImageView.__init__(parent=None)

- Location: widgets/image_view.py
- What it does: Initializes the viewer and crop state.
- How it works: It creates the scene and pixmap item, enables drag-and-drop, configures zoom/pan behavior, and initializes crop overlay state.
- Output / effect: Creates the viewer.

ImageView.set_image(pixmap)

- Location: widgets/image_view.py
- What it does: Displays a new pixmap in the scene.
- How it works: It replaces the current pixmap, updates the scene rect, and repaints the viewport.
- Output / effect: Shows the new image.

ImageView.fit_image()

- Location: widgets/image_view.py
- What it does: Fits the current image into the viewport.
- How it works: It calls fitInView while preserving aspect ratio.
- Output / effect: Adjusts the view transform.

ImageView.zoom_in()

- Location: widgets/image_view.py
- What it does: Zooms into the image.
- How it works: It scales the current view transform by a fixed factor larger than one.
- Output / effect: Increases magnification.

ImageView.zoom_out()

- Location: widgets/image_view.py
- What it does: Zooms out from the image.
- How it works: It scales the current view transform by the inverse zoom factor.

- Output / effect: Reduces magnification.

ImageView.set_crop_mode(enabled)

- Location: widgets/image_view.py
- What it does: Turns crop-editing mode on or off.
- How it works: It switches between crop interaction and normal hand-drag panning.
- Output / effect: Changes viewer interaction mode.

ImageView.set_crop_lock(enabled, ratio)

- Location: widgets/image_view.py
- What it does: Updates crop aspect-lock settings.
- How it works: It stores whether aspect locking is active and the ratio value to enforce during crop drags.
- Output / effect: Changes crop constraints.

ImageView.set_crop_rect(rect)

- Location: widgets/image_view.py
- What it does: Loads a crop rectangle into committed and staged storage.
- How it works: It stores the rectangle in both forms so editing can continue from the current crop state.
- Output / effect: Updates the crop overlay.

ImageView.clear_crop_rect()

- Location: widgets/image_view.py
- What it does: Clears the crop rectangle.
- How it works: It removes both committed and staged crop rectangles and repaints.
- Output / effect: Removes crop state.

ImageView.current_crop_rect()

- Location: widgets/image_view.py
- What it does: Returns the current staged crop rectangle.
- How it works: It normalizes the staged rectangle and rounds values to integer coordinates.
- Output / effect: Returns a QRect.

ImageView.commit_staged_crop()

- Location: widgets/image_view.py
- What it does: Promotes the staged crop rectangle to committed state.
- How it works: It copies the staged rectangle into committed storage and repaints.

- Output / effect: Stores the current crop box.

ImageView.revert_staged_crop()

- Location: widgets/image_view.py
- What it does: Restores the staged crop rectangle from the committed one.
- How it works: It discards uncommitted crop changes and repaints.
- Output / effect: Cancels the temporary crop edit.

ImageView.dragEnterEvent(event)

- Location: widgets/image_view.py
- What it does: Accepts supported dragged image files.
- How it works: It checks incoming URLs and only accepts PNG, JPG, and JPEG files.
- Output / effect: Controls drag acceptance.

ImageView.dragMoveEvent(event)

- Location: widgets/image_view.py
- What it does: Keeps a supported drag operation active while moving.
- How it works: It accepts proposed actions for URL-based drags.
- Output / effect: Maintains drag-and-drop behavior.

ImageView.dropEvent(event)

- Location: widgets/image_view.py
- What it does: Emits the dropped image path when a supported file is released into the viewer.
- How it works: It checks dropped URLs, validates the extension, and emits imageDropped for the first supported file.
- Output / effect: Hands the path back to the main window.

ImageView.wheelEvent(event)

- Location: widgets/image_view.py
- What it does: Zooms the view with the mouse wheel.
- How it works: Positive wheel movement zooms in and negative movement zooms out, then the viewport is updated.
- Output / effect: Changes viewer magnification.

ImageView.drawForeground(painter, rect)

- Location: widgets/image_view.py
- What it does: Draws the crop overlay, guides, and drag handles.

- How it works: It darkens the outside area, outlines the crop region, draws rule-of-thirds guides, and places edge, corner, and center handles scaled to the current zoom level.
- Output / effect: Renders crop UI over the preview.

ImageView._scene_pos(event)

- Location: widgets/image_view.py
- What it does: Converts a mouse event into scene coordinates.
- How it works: It maps viewport-space positions into the graphics-scene coordinate system.
- Output / effect: Returns a QPointF.

ImageView._pick_handle(scene_pos)

- Location: widgets/image_view.py
- What it does: Determines which crop handle or move zone is under the cursor.
- How it works: It checks all stored handle rectangles first and falls back to move mode when the cursor is inside the crop rectangle.
- Output / effect: Returns a handle name or None.

ImageView._clamp_crop(rect)

- Location: widgets/image_view.py
- What it does: Constrains a crop rectangle to valid image bounds and minimum size.
- How it works: It normalizes the rectangle, enforces the minimum crop width and height, and pushes it back inside pixmap bounds when needed.
- Output / effect: Returns a corrected QRectF.

ImageView._apply_aspect_to_corner(base, moving_corner, scene_pos)

- Location: widgets/image_view.py
- What it does: Builds a new crop rectangle for a corner drag when aspect lock is active.
- How it works: It keeps the opposite corner anchored and adjusts the dragged corner so the crop still matches the requested aspect ratio.
- Output / effect: Returns an aspect-constrained QRectF.

ImageView._update_crop_from_handle(scene_pos)

- Location: widgets/image_view.py
- What it does: Updates the staged crop rectangle during drag operations.
- How it works: It interprets the active handle, applies movement, optionally enforces aspect lock, and clamps the result to valid bounds.
- Output / effect: Updates crop state in real time.

ImageView.mousePressEvent(event)

- Location: widgets/image_view.py
- What it does: Begins a crop drag or passes the event to the base class.
- How it works: In crop mode it creates a default crop box if needed, picks the clicked handle, or starts a new crop from the click position.
- Output / effect: Starts crop editing.

ImageView.mouseMoveEvent(event)

- Location: widgets/image_view.py
- What it does: Updates crop interaction while dragging.
- How it works: If a handle is active it recalculates the crop rectangle, emits cropPreviewChanged, and repaints the viewport.
- Output / effect: Performs live crop updates.

ImageView.mouseReleaseEvent(event)

- Location: widgets/image_view.py
- What it does: Ends crop dragging.
- How it works: It clears the active handle and repaints when the left mouse button is released in crop mode.
- Output / effect: Finishes the gesture.

main_window.py

This file is the orchestration layer that binds state, widgets, rendering, history, and persistence together.

Class MainWindow

- Location: main_window.py
- What it does: Acts as the main application controller and top-level window.
- How it works: It owns the source image, preview image, adjustment state, history manager, worker thread, tabs, widgets, actions, and file operations. Most package components are coordinated from here.
- Output / effect: Central application object.

MainWindow.__init__()

- Location: main_window.py
- What it does: Initializes the main window and runtime state.
- How it works: It sets up session variables, history, timers, worker-thread connections, then builds the UI and applies styling.
- Output / effect: Creates the main application window.

MainWindow.closeEvent(event)

- Location: main_window.py
- What it does: Handles application shutdown.
- How it works: It asks whether unsaved changes should be saved or discarded, and shuts down the render worker thread cleanly before closing.
- Output / effect: Accepts or rejects the close event.

MainWindow.build_ui()

- Location: main_window.py
- What it does: Constructs the visible interface.
- How it works: It creates the central splitter, the tab widget, the image viewer, toolbar actions, and all tab pages.
- Output / effect: Builds the main layout.

MainWindow.apply_styles()

- Location: main_window.py
- What it does: Applies the application's dark visual theme.
- How it works: It sets a Qt stylesheet that controls general colors, borders, buttons, sliders, and tab appearance.
- Output / effect: Defines visual styling.

MainWindow.build_adjust_tab()

- Location: main_window.py
- What it does: Creates the Adjustments tab.
- How it works: It builds grouped controls for scalar grading, channel intensity, tonal controls, and reset options inside a scrollable panel.
- Output / effect: Returns the tab widget.

MainWindow.build_curve_tab()

- Location: main_window.py
- What it does: Creates the Curves tab.
- How it works: It adds the active-channel selector, reset button, and CurveEditor widget, then wires the live-edit signals.
- Output / effect: Returns the tab widget.

MainWindow.build_histogram_tab()

- Location: main_window.py

- What it does: Creates the Histogram tab.
- How it works: It places the standalone histogram widget and explanatory text inside a simple vertical layout.
- Output / effect: Returns the tab widget.

MainWindow.build_transform_tab()

- Location: main_window.py
- What it does: Creates the Transform tab.
- How it works: It builds navigation buttons, rotate/flip actions, crop controls, and resize controls.
- Output / effect: Returns the tab widget.

MainWindow.create_actions()

- Location: main_window.py
- What it does: Creates reusable QAction objects and shortcuts.
- How it works: It defines actions for open, export, reset, undo, redo, zooming, before/after preview, and project/preset operations.
- Output / effect: Populates action objects for the toolbar and shortcuts.

MainWindow.create_toolbar()

- Location: main_window.py
- What it does: Builds the top toolbar.
- How it works: It adds file, history, project, preset, fit, and before/after actions in a compact top-row layout.
- Output / effect: Creates the main toolbar.

MainWindow.make_single_slider(mn, mx, default)

- Location: main_window.py
- What it does: Creates one slider and its numeric label.
- How it works: It configures the slider range and default value, then keeps the label synced to the slider value.
- Output / effect: Returns a slider/label pair.

MainWindow.wrap_slider_row(slider, label)

- Location: main_window.py
- What it does: Places a slider and its value label into one row widget.
- How it works: It creates a helper container so sliders can be inserted consistently into layouts.
- Output / effect: Returns a QWidget row.

MainWindow.make_slider_group(parent_layout, spec_list)

- Location: main_window.py
- What it does: Builds a series of labeled sliders from a specification list.
- How it works: It loops over the field specifications, creates one slider row per item, connects the change callback, and returns a lookup dictionary keyed by control name.
- Output / effect: Returns a control dictionary.

MainWindow.make_tone_group(title, prefix)

- Location: main_window.py
- What it does: Builds a shadows, midtones, or highlights control group.
- How it works: It creates separate R, G, and B sliders for one tonal range and adds a dedicated reset button.
- Output / effect: Returns a QGroupBox.

MainWindow.get_display_target_size()

- Location: main_window.py
- What it does: Computes the preview render size for the current viewport.
- How it works: It determines the current geometry size, substitutes resize dimensions when resize is active, and then fits the result into the viewer using the preview oversampling scale.
- Output / effect: Returns a width/height tuple.

MainWindow.get_crop_reference_size()

- Location: main_window.py
- What it does: Computes the coordinate space used for crop editing.
- How it works: It clones the current state, disables crop, applies rotation and flips, and returns the transformed image size before crop so the crop overlay matches what the user sees.
- Output / effect: Returns a width/height tuple.

MainWindow.get_current_crop_ratio()

- Location: main_window.py
- What it does: Returns the selected crop aspect ratio.
- How it works: It resolves the Original option from the current image or crop and parses fixed ratios such as 16:9 from the combo-box text.
- Output / effect: Returns a float ratio.

MainWindow.update_crop_lock()

- Location: main_window.py

- What it does: Pushes current crop-lock settings into the ImageView.
- How it works: It updates the viewer with the active lock flag and ratio and refreshes the staged crop box if crop mode is on.
- Output / effect: Synchronizes crop behavior.

MainWindow.refresh_staged_crop_aspect()

- Location: main_window.py
- What it does: Reapplies aspect-lock rules to the staged crop rectangle.
- How it works: It changes width or height around the crop center so the visible crop box matches the selected ratio.
- Output / effect: Updates the visible crop box.

MainWindow.geometry_rect_to_display_rect(rect)

- Location: main_window.py
- What it does: Converts a crop rectangle from geometry coordinates into display coordinates.
- How it works: It scales the stored crop rectangle from transformed image space into the current viewer-space preview size.
- Output / effect: Returns a display-space QRect.

MainWindow.display_rect_to_geometry_rect(rect)

- Location: main_window.py
- What it does: Converts a crop rectangle from viewer coordinates back into geometry coordinates.
- How it works: It scales the rectangle from display space into transformed image space and clamps the result to valid bounds.
- Output / effect: Returns a geometry-space QRect.

MainWindow.sync_viewer_crop_rect()

- Location: main_window.py
- What it does: Synchronizes the visible crop overlay with the stored state.
- How it works: It converts the active crop rectangle into display space or creates a default crop box when no crop is currently enabled.
- Output / effect: Updates the viewer crop UI.

MainWindow.request_fast_render(skip_tonal)

- Location: main_window.py
- What it does: Queues a debounced fast render.

- How it works: It stores a clone of the current state and restarts the short debounce timer instead of rendering immediately on every micro-change.
- Output / effect: Schedules a fast preview request.

MainWindow.flush_fast_render_request()

- Location: main_window.py
- What it does: Submits the pending fast render to the worker.
- How it works: It increments the render generation counter and creates a RenderRequest configured for fast preview rendering.
- Output / effect: Starts background preview work.

MainWindow.request_full_render()

- Location: main_window.py
- What it does: Submits a full-quality render request.
- How it works: It increments the generation counter and sends a request that also refreshes histogram data.
- Output / effect: Starts the final-quality preview pass.

MainWindow.update_viewer_pixmap(show_original=False)

- Location: main_window.py
- What it does: Updates the image shown in the viewer.
- How it works: It picks either the original or edited preview array, computes a lightweight content key to avoid redundant refreshes, converts the array to a pixmap, and optionally fits the image after a new load.
- Output / effect: Refreshes the displayed preview.

MainWindow.apply_histogram_to_widgets(hist)

- Location: main_window.py
- What it does: Pushes histogram data into both histogram displays.
- How it works: It stores the latest histogram and updates the standalone histogram widget and the curve editor overlay.
- Output / effect: Synchronizes histogram UI.

MainWindow.confirm_discard_unsaved()

- Location: main_window.py
- What it does: Asks the user what to do with unsaved changes.
- How it works: It shows a warning dialog with Save Project, Discard, and Cancel choices and returns whether the caller should continue.

- Output / effect: Returns True when the operation may proceed.

MainWindow.resizeEvent(event)

- Location: main_window.py
- What it does: Responds to main-window resize events.
- How it works: It refreshes resize-control reflection of the current geometry state and keeps related controls synchronized. In this version it is part of the broader UI refresh path after geometry changes.
- Output / effect: Refreshes UI state on resize.

MainWindow.sync_curve_editor_from_state()

- Location: main_window.py
- What it does: Makes the curve editor reflect the current state.
- How it works: It updates the active channel, point list, and histogram background in the CurveEditor widget.
- Output / effect: Refreshes curve UI.

MainWindow.refresh_actions()

- Location: main_window.py
- What it does: Updates enabled/disabled action states.
- How it works: It enables export only when an image is loaded and toggles undo/redo actions based on history availability.
- Output / effect: Keeps action state accurate.

MainWindow.on_slider_changed(key)

- Location: main_window.py
- What it does: Handles changes to scalar grading sliders.
- How it works: It maps the slider value back into the correct parameter range, clones the adjustment state, updates the selected field, and commits the change without immediately creating a final history checkpoint.
- Output / effect: Updates live scalar grading.

MainWindow.on_tone_slider_changed(key)

- Location: main_window.py
- What it does: Handles changes to shadows, midtones, and highlights sliders.
- How it works: It identifies which tonal group and channel changed, updates the cloned state, and commits the result for live preview.
- Output / effect: Updates tonal-region grading.

MainWindow.on_curve_channel_changed(channel)

- Location: main_window.py
- What it does: Changes the active curve channel.
- How it works: It stores the channel name and refreshes the curve editor to show the matching point set.
- Output / effect: Updates editing context.

MainWindow.on_curve_points_changed(points)

- Location: main_window.py
- What it does: Handles live point edits from the curve editor.
- How it works: It writes the new point list into the active curve on a cloned state and commits it for preview rendering.
- Output / effect: Updates curve state.

MainWindow.on_crop_preview_changed(rect)

- Location: main_window.py
- What it does: Shows crop-size feedback in the status bar.
- How it works: It formats the current crop width and height as a short status message.
- Output / effect: Updates the status bar.

MainWindow.apply_crop_from_view()

- Location: main_window.py
- What it does: Commits the crop box shown in the viewer to the stored state.
- How it works: It converts the staged display-space crop rectangle into geometry-space coordinates, stores it in the state, commits that state, and exits crop mode.
- Output / effect: Applies the crop.

MainWindow.cancel_crop_edit()

- Location: main_window.py
- What it does: Cancels the current crop-edit session.
- How it works: It tells the viewer to restore the staged crop rectangle from the last committed one and exits crop mode.
- Output / effect: Reverts uncommitted crop changes.

MainWindow.on_resize_dimension_changed(changed_axis)

- Location: main_window.py
- What it does: Handles resize width and height slider changes.

- How it works: It rebuilds the pre-resize geometry image, uses that as the true aspect-ratio reference, optionally updates the opposite dimension when aspect lock is enabled, and then commits the resize control state.
- Output / effect: Updates resize interactively.

MainWindow.on_resize_controls_changed()

- Location: main_window.py
- What it does: Commits the current resize controls into AdjustmentState.
- How it works: It copies the enabled flag and dimensions into a cloned state and commits the result.
- Output / effect: Updates resize state.

MainWindow.undo()

- Location: main_window.py
- What it does: Restores the previous history state.
- How it works: It finalizes any active interaction, asks the history manager for the previous snapshot, synchronizes the UI, and refreshes rendering.
- Output / effect: Performs undo.

MainWindow.redo()

- Location: main_window.py
- What it does: Restores the next history state.
- How it works: It asks the history manager for the redo snapshot, synchronizes the UI, and refreshes rendering.
- Output / effect: Performs redo.

MainWindow.reset_all()

- Location: main_window.py
- What it does: Clears all edits.
- How it works: It creates a default AdjustmentState and commits it when an image is loaded.
- Output / effect: Resets the whole session state.

MainWindow.reset_current_tab()

- Location: main_window.py
- What it does: Resets only the category represented by the active tab.
- How it works: It inspects the current tab name and clears only adjustment, curve, or transform-related fields while preserving the rest of the session.
- Output / effect: Performs a scoped reset.

MainWindow.reset_channel(channel)

- Location: main_window.py
- What it does: Clears one RGB intensity channel.
- How it works: It sets the chosen red, green, or blue intensity field back to zero and commits the update.
- Output / effect: Resets one channel offset.

MainWindow.reset_tone_group(prefix)

- Location: main_window.py
- What it does: Clears one tonal-region RGB group.
- How it works: It replaces the chosen shadows, midtones, or highlights object with a zeroed ToneRGB instance and commits the change.
- Output / effect: Resets one tonal group.

MainWindow.reset_current_curve()

- Location: main_window.py
- What it does: Resets the currently selected curve to the identity line.
- How it works: It replaces the active curve point list with the two standard endpoints (0,0) and (1,1).
- Output / effect: Clears the selected curve.

MainWindow.clear_crop()

- Location: main_window.py
- What it does: Disables cropping and removes the crop overlay.
- How it works: It resets crop state, commits the change, and clears the viewer crop rectangle.
- Output / effect: Removes active crop state.

MainWindow.reset_resize()

- Location: main_window.py
- What it does: Disables resizing and clears resize dimensions from state.
- How it works: It replaces the resize state with a default disabled ResizeState and commits the update.
- Output / effect: Removes active resize state.

MainWindow.remap_crop_for_new_transform(old_state, new_state)

- Location: main_window.py
- What it does: Keeps crop targeting consistent when rotation or flip changes.
- How it works: It maps the old crop rectangle from transformed coordinates back to original-image coordinates and then forward into the new transformed coordinate system.

- Output / effect: Preserves crop meaning across transforms.

MainWindow.remap_resize_for_new_transform(old_state, new_state, delta_rotation=0)

- Location: main_window.py
- What it does: Keeps resize dimensions logically consistent across transform changes.
- How it works: For 90-degree and 270-degree rotations it swaps width and height so a stretched output remains correctly oriented. For flips and 0/180-degree changes it keeps the dimensions as-is.
- Output / effect: Preserves resize meaning across transforms.

MainWindow.sync_controls_from_state()

- Location: main_window.py
- What it does: Refreshes the visible controls from the current AdjustmentState.
- How it works: It writes scalar values into sliders, rebuilds resize controls from the current geometry base image, updates curve state, and refreshes crop-lock information.
- Output / effect: Keeps the UI in sync with state.

MainWindow.rotate_image(delta)

- Location: main_window.py
- What it does: Applies a 90-degree incremental rotation.
- How it works: It clones the current state, updates the rotation, remaps crop and resize consistency, and commits the new state.
- Output / effect: Rotates the session.

MainWindow.toggle_flip(axis)

- Location: main_window.py
- What it does: Toggles horizontal or vertical mirroring.
- How it works: It flips the relevant boolean flag, remaps crop and resize behavior, and commits the updated state.
- Output / effect: Mirrors the session.

MainWindow.toggle_before_after()

- Location: main_window.py
- What it does: Switches between edited preview and original preview.
- How it works: It toggles the preview flag and refreshes the viewer pixmap accordingly.
- Output / effect: Changes comparison mode.

install_slider_commit_hooks(window)

- Location: main_window.py

- What it does: Attaches press/release hooks to sliders so interactions finalize cleanly.
- How it works: It iterates over grading and resize sliders, marks the window as actively dragging on slider press, and calls finalize_interaction on release.
- Output / effect: Improves render timing and history behavior.

main.py
This file is the normal startup entry point for the packaged application.

main()
- Location: main.py
- What it does: Starts the Qt application.
- How it works: It creates QApplication, sets the application name, builds MainWindow, installs slider commit hooks, shows the window, and enters the event loop.
- Output / effect: Launches the program.

__main__.py
This file is the package-execution shim.

Package launcher block
- Location: __main__.py
- What it does: Supports running the package with python -m color_grading_app_pkg.
- How it works: It imports main from main.py and calls it when the package is executed as a module.
- Output / effect: Provides package-style startup.

__init__.py
This file marks the top-level package.

Package marker file
- Location: __init__.py
- What it does: Marks the top-level folder as a Python package.
- How it works: In the current version it contains no runtime logic.
- Output / effect: No runtime methods.

widgets/__init__.py
This file marks the widgets subpackage.

Subpackage marker file
- Location: widgets/__init__.py
- What it does: Marks the widgets folder as a Python subpackage.
- How it works: In the current version it contains no runtime logic.

- Output / effect: No runtime methods.